

# ServiceNow VR, CC and CVR with Tenable and Wiz

How Vulnerability Response, Configuration Compliance and Container Vulnerability Response work, and exactly how Tenable and Wiz feed each one, from the vendors' official documentation.

## 3 engines

Separate finding models, one shared rule pipeline

## Scanner closes

Owners can only Resolve; unreported items go stale

## Critical + High

Tenable's default import filter

## 4 keys

Effectively permanent once data lands

## At a glance

12

original figures

3

separate engines

4

keys to lock before go-live

25

documented contradictions

| Topic                        | The fact to state                                                                                                                                               | Confidence                     |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------|
| <b>Three engines</b>         | VR (hosts), CC (configuration) and CVR (containers) keep separate finding models but share one rule pipeline: CI lookup, risk, assignment, task rules, targets. | <a href="#">Official docs</a>  |
| <b>Closure</b>               | Only the scanner closes work. Manual Close was removed; owners can only Resolve. Unreported items go stale after 90 days by default.                            | <a href="#">Official docs</a>  |
| <b>VI key</b>                | A VI is unique per CI + vulnerability + integration instance, so Tenable and Wiz each create a VI for the same CVE on the same server.                          | <a href="#">Official docs</a>  |
| <b>CC</b>                    | CC imports pass/fail verdicts it never evaluates. Its base task rule, assignment rule and reapply job ship disabled.                                            | <a href="#">Official docs</a>  |
| <b>CVR</b>                   | CVR anchors findings on the source image. CVITs cannot be closed manually; the documented default key is repository + vulnerability + image.                    | <a href="#">Official docs</a>  |
| <b>Tenable coverage</b>      | Tenable VM feeds VR and CC. Security Center feeds VR only. Tenable Cloud Security feeds VR (cloud hosts) and CVR.                                               | <a href="#">Official docs</a>  |
| <b>Tenable defaults</b>      | Critical and High only. Fixed findings update VIs but create none. The Tenable Risk Rule ships off.                                                             | <a href="#">Official docs</a>  |
| <b>Wiz coverage</b>          | One ServiceNow-built app feeds VR, CC, CVR, Application VR and AI Security, and lists CC and CVR as prerequisites even for host-only use.                       | <a href="#">Official docs</a>  |
| <b>Version fork</b>          | Keep VR, CC, CVR and scanner apps on the same side of v30. September 2026: Tenable 30.5.3 / 6.2.3, Wiz 32.8.4 / 4.2.4, CVR 30.8.5 / 2.20.3.                     | <a href="#">Release notes</a>  |
| <b>Irreversible choices</b>  | VI port key, Wiz detection key, Wiz CVIT key and Tenable CC test key; Wiz AI posture routing cannot be switched off once saved.                                 | <a href="#">Official docs</a>  |
| <b>Documentation quality</b> | The Brazil pages contradict themselves on about 25 points that affect configuration.                                                                            | <a href="#">Official docs</a>  |
| <b>Wiz handover</b>          | ServiceNow's Wiz app reached GA on July 31, 2025, replacing three Wiz-built Store apps.                                                                         | <a href="#">Community only</a> |

**Bottom line.** The integration problem is about keys and filters, not connectors. Before the first import, decide what makes a finding unique, which severities and resource types enter, which scanner owns which asset, and which signal is canonical.

**How confidence is labeled.** [Official docs](#) means ServiceNow's documentation repository on GitHub (brazil branch, pages dated September 2026). [Release notes](#) means ServiceNow Store or family release notes outside that repository. [Community only](#) means ServiceNow Community posts, often written by ServiceNow staff but not product documentation. [Third-party](#) means vendor, analyst, legal or news sources. [Inference](#) marks the author's own reasoning from documented behavior.

## How to talk about it accurately

These are the statements most likely to be said loosely in a meeting, with the accurate version to use instead.

| Avoid saying                                               | Say instead                                                                                                      |
|------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| "Tenable and Wiz findings merge into one vulnerable item." | "Each integration creates its own VI. The same CVE on the same server from both scanners is two VIs."            |
| "Owners close remediation tasks when they patch."          | "Owners mark work Resolved. The next scan closes it, or it goes stale."                                          |
| "CC checks our servers against CIS."                       | "CC imports the scanner's pass/fail verdicts. Tenable or Wiz does the checking."                                 |
| "We get all of Tenable's findings."                        | "By default only Critical and High arrive. Medium has to be switched on."                                        |
| "Security Center feeds compliance too."                    | "Only Tenable VM has a documented CC path. Security Center is vulnerability-only."                               |
| "Wiz container findings attach to running containers."     | "CVR anchors them on the source image and adds deployment context from Wiz."                                     |
| "We can change the keys later."                            | "Four keys are effectively permanent once data lands. We decide them first."                                     |
| "A host-only Wiz rollout needs just VR."                   | "The Wiz app lists CC and CVR as prerequisites, each a separate subscription."                                   |
| "Let's turn on AI posture routing to try it."              | "Once saved it cannot be turned off. We wait until AI Security governance is ready."                             |
| "The docs are the spec."                                   | "The docs contradict themselves on about 25 points. The instance's transform maps and key records are the spec." |

**Three facts worth memorizing.** First, only the scanner closes work. Second, Tenable and Wiz each create their own vulnerable item for the same CVE on the same server. Third, Tenable imports only Critical and High findings by default.

# Contents

---

1. Summary: Tenable and Wiz feed three separate engines
2. Three finding models share one rule pipeline and one closure rule
3. Separate subscriptions and the v30 fork decide what you can run
4. The integration matrix shows Tenable narrowing and Wiz spanning
5. Host findings reach VR through a Fixed-then-Open Tenable chain and a UUID-keyed Wiz feed
6. Configuration verdicts reach CC only from Tenable VM and Wiz
7. Container data reaches CVR through Tenable.cs or a three-step Wiz chain
8. Least-privilege setup starts on the vendor side
9. Two dozen documentation contradictions touch design decisions
10. A large-enterprise program should decide keys, ownership and filters before data lands
11. Conclusion
12. Appendix A. Questions to put to ServiceNow, Tenable and Wiz
13. Appendix B. Glossary

## Figures

Figure 1. Four Tenable and Wiz sources, three ServiceNow engines, one shared pipeline.

Figure 2. The shared processing order, and how closure, staleness, SLA clocks and base risk differ per engine.

Figure 3. VR object model: detection, vulnerable item and remediation task, with state roll-up rules.

Figure 4. CC object model: scanner-supplied tests, imported results, and remediation tasks.

Figure 5. CVR object model, the CVIT key, and every way a CVIT closes.

Figure 6. The v30 fork: classic and USEM tracks must match across every app.

Figure 7. Tenable host paths into VR: job order, defaults and CI lookup rules per Tenable product.

Figure 8. Tenable compliance path into CC, the backfill, and the test-key decision.

Figure 9. Wiz paths into CC, rejected-finding options, and the one-way AI posture switch.

Figure 10. Container paths into CVR: the Tenable Cloud Security chain and the Wiz deployment-context chain.

Figure 11. Least-privilege setup across Tenable, ServiceNow and Wiz.

Figure 12. Decide these before data lands.

**About the figures.** Every figure is an original diagram drawn from the cited documentation. Vendor screenshots are not reproduced because they are copyrighted. If you need product screenshots for a presentation, capture them from your own instance.

# ServiceNow VR, CC and CVR with Tenable and Wiz

## Summary: Tenable and Wiz feed three separate engines

---

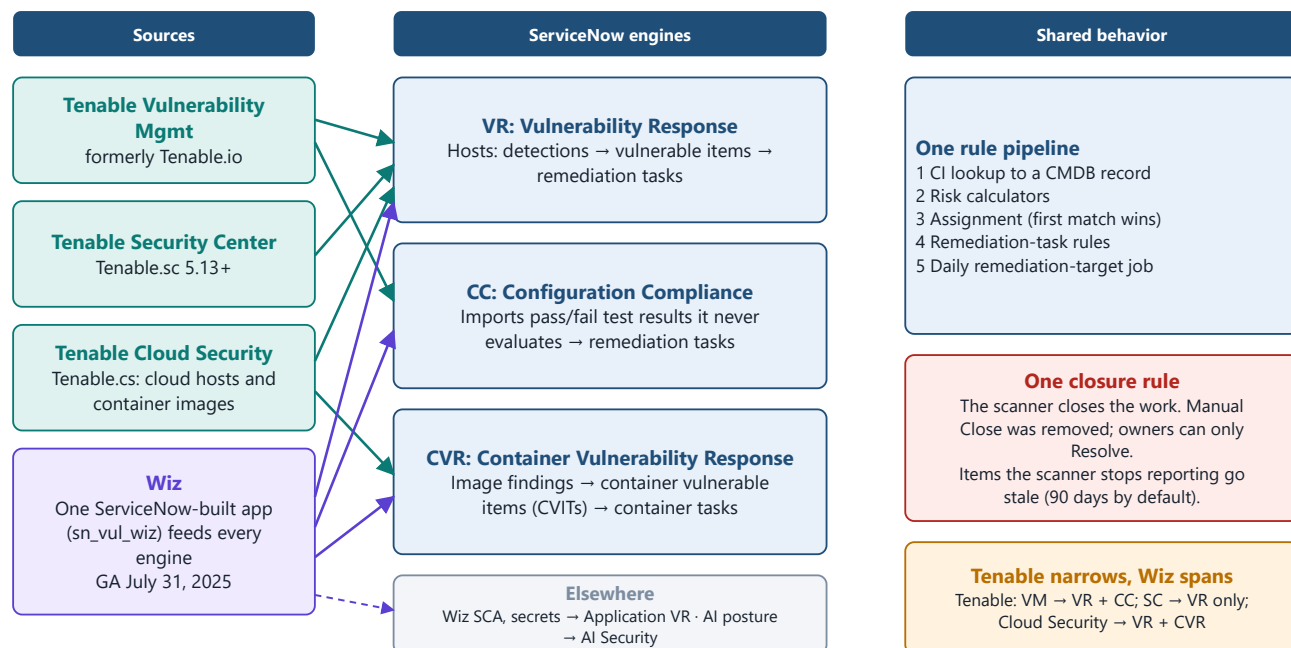
ServiceNow Vulnerability Response (VR), Configuration Compliance (CC) and Container Vulnerability Response (CVR) are **three separate data models that share one remediation pipeline**. VR rolls scanner **detections** into vulnerable items (VIs) and remediation tasks. CC imports pass/fail **configuration test results** that it never evaluates itself. CVR attaches container image **findings** to the source Docker image, not to running containers. In all three the scanner, not the remediation owner, closes the work: the manual Close actions have been removed, and 90-day stale rules sweep up whatever the scanner stops reporting. Tenable feeds the three engines through one ServiceNow-built, separately subscribed Store app, and its coverage is uneven. **Tenable Vulnerability Management (TVM) feeds VR and CC. Tenable Security Center feeds VR only. Tenable Cloud Security (Tenable.cs) is the only Tenable source for CVR.** Wiz feeds all three, plus Application VR, from one service account. Since July 2025 that has run through a ServiceNow-built app that replaced three Wiz-built ones, and its installer requires VR, CC and CVR to be present even for a host-only rollout. USEM (VR v30 and later) moves the rules into shared `sn_sec_*` tables but keeps separate VR, CC and CVR finding tables. ServiceNow tells customers who are not moving to USEM to keep every one of these apps, scanner integrations included, below v30. Several decisions are hard to reverse and belong before the first import:

- the VI, detection, container vulnerable item (CVIT) and Tenable compliance-test keys;
- severity filters, which default to Critical and High only;
- which scanner owns which asset.

The Brazil documentation contradicts itself on more than twenty points that bear on those decisions. Validate every mapping below on a sub-production instance before it drives an SLA or a dashboard.

**How the sourcing works.** ServiceNow's official documentation repository, `brazil` branch, is the primary source; most pages cited carry `last_updated: 2026-09-10`. Citations take the form `https://github.com/ServiceNow/ServiceNowDocs/blob/brazil/<path>`. ServiceNow's Store release notes on `servicenow.com` are also official. They are more current (September 2026) but sit outside the repo, and they were read through a summarizing fetch, so check exact wording there before quoting it. Vendor-side facts come from Tenable's documentation (`docs.tenable.com`, `developer.tenable.com`) and Wiz's public website. Wiz's product documentation (`docs.wiz.io`) requires a login, so Wiz-side requirements rest mostly on ServiceNow's pages. Anything from the ServiceNow Community or another third party is labeled as such. A companion report, *ServiceNow USEM, Tenable and Wiz*, covers the USEM layer itself: the workspace, unified scoring defaults, cross-scanner deduplication and packaging tiers. This report repeats that material only where VR, CC or CVR cannot be explained without it.

**Figure 1. Four Tenable and Wiz sources, three ServiceNow engines, one shared pipeline.**



Source: Official docs: [tenable-io / sc / cs-integrations-list](#), [Wiz integration pages](#), [CC and VR assignment and task pages](#).

## Three finding models share one rule pipeline and one closure rule

All three engines process a new or reopened finding the same way:

1. Scanner data lands on a discovered item (or, in CVR, a discovered container image), which CI lookup rules resolve to a CMDB record.
2. Risk calculators score the finding.
3. Assignment rules run by execution order, and the first match wins.
4. Remediation-task (RT) rules group findings into tasks.
5. A daily job sets remediation targets.

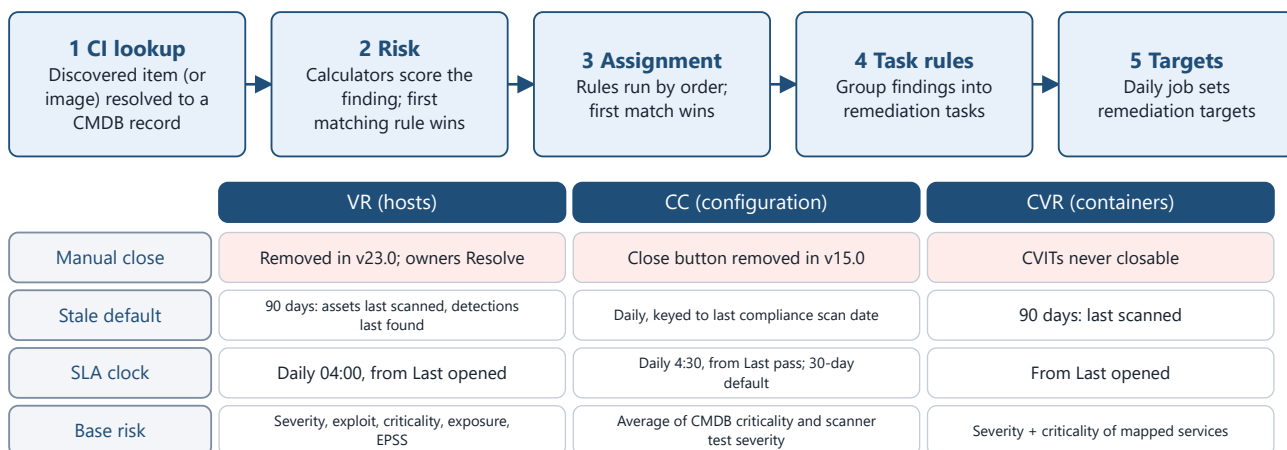
CC documents the order explicitly: CI matching, then risk, then assignment, then RT rules ([CC assignment rules](#); [CC remediation tasks](#)). VR uses the same first-match assignment logic ([VR assignment rules](#)) and a daily 04:00 target job ([VR remediation targets](#)). What differs is what counts as a finding, how findings are keyed, and what they attach to.

| Layer        | VR (hosts)                                                                                                                                                                                            | CC (configuration)                                                               | CVR (containers)                                                                                   |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| Asset anchor | Discovered item <code>sn_sec_cm_n_src_ci</code> → CI. With no match, IRE creates <code>cmdb_ci_incomplete_ip</code> , <code>cmdb_ci_unclassified_hardware</code> or <code>cmdb_ci_cmp_resource</code> | Discovered item → CI. With no match, falls back to an "Unmatched CI" placeholder | Discovered container image <code>sn_vul_container_image</code> → <code>cmdb_ci_docker_image</code> |

| Layer                 | VR (hosts)                                                                                                  | CC (configuration)                                                                                                                                                    | CVR (containers)                                                                                |
|-----------------------|-------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Knowledge record      | Third-party entry (TPE)<br><code>sn_vul_third_party_entry</code> , e.g. TEN- plugin IDs, linked to NVD CVEs | Test group <code>sn_vulc_policy</code> ; test <code>sn_vulc_test</code> ; authoritative source <code>sn_vulc_auth_src</code> ; citation <code>sn_vulc_citation</code> | TPE / <code>sn_vul_entry</code> , plus image layer and package tables                           |
| Raw observation       | Detection <code>sn_vul_detection</code>                                                                     | Test result <code>sn_vulc_result</code> , with history in <code>sn_vulc_result_history</code>                                                                         | Image finding <code>sn_vul_container_image_findings</code>                                      |
| Work item             | VI <code>sn_vul_vulnerable_item</code>                                                                      | The test result itself                                                                                                                                                | CVIT <code>sn_vul_container_image_vulnerable_item</code>                                        |
| Grouping              | RT <code>sn_vul_vulnerability</code>                                                                        | RT <code>sn_vulc_result_group</code>                                                                                                                                  | Container RT (CVUL) <code>sn_vul_container_vulnerability</code>                                 |
| Default work-item key | CI + vulnerability + integration instance. Port is optional                                                 | CI + technology + test (history key). Tenable can add keys                                                                                                            | Image repository + vulnerability + image. Registry, cluster, namespace and service are optional |
| Manual close          | Removed in v23.0. Owners can only Resolve                                                                   | Close button removed in v15.0                                                                                                                                         | CVITs never closable. CVUL Close removed in v2.10                                               |
| Stale default         | 90 days, on assets last scanned and on detections last found                                                | Daily job keyed to the item's last compliance scan date                                                                                                               | 90 days, "Container Vulnerabilities last scanned"                                               |
| SLA clock             | Daily 04:00, from Last opened                                                                               | Daily 4:30, from Last pass, 30-day default                                                                                                                            | From Last opened                                                                                |
| Base risk             | Severity, exploit, criticality, exposure and EPSS                                                           | Average of CMDB business criticality and scanner test criticality                                                                                                     | Severity, plus business criticality from the image's services                                   |

Sources for the table: [VR detections](#); [VI key](#); [RT states](#); [VR auto-close rules](#); [IRE classes](#); [CC tables](#); [CC test results](#); [CC states](#); [CC auto-close](#); [CC targets](#); [CC risk](#); [CVR tables](#); [CVIT key](#); [CVR states](#); [CVR auto-close](#); [CVR dashboard](#).

**Figure 2. The shared processing order, and how closure, staleness, SLA clocks and base risk differ per engine.**



Source: Official docs: cc-assignment-rules, cc-groups, create-vul-assign-rules, vr-rt-states, create-auto-close-rules, CC and CVR risk pages.

## VR turns detections into vulnerable items, then into remediation tasks

A detection is "a single, distinct occurrence of a vulnerability as reported by a scanner." Each detection is keyed by a hashed, integration-specific **detection key**:

- **Tenable and Qualys**: vulnerability + port + protocol + asset ID, with no proof.
- **Rapid7**: adds proof and NIC.
- **Default** (when no key is specified): adds proof ([VR detections](#)).

Detections collapse into a **VI that is unique per CI + vulnerability + integration instance**. The "Include port" option creates one VI per port, but **once you enable it, you must delete all VR data before you can disable it again** ([VI key](#)). Because the integration instance is part of the key, and ServiceNow states that VIs are not de-duplicated across integrations, **the same CVE on the same server reported by both Tenable and Wiz produces two VIs** ([VR integrations](#)). The companion USEM study covers the narrow duplicate-VI tooling that exists. Plan cross-scanner reporting around CVE + CI, not VI counts.

Closure belongs to the scanner. RTs move through Open, Under Investigation, Deferred (shown as In Review during approval), Awaiting Implementation, Resolved and Closed. **The Close button was removed in v23.0, so "closure of a remediation task is driven by the scanner,"** and owners can only mark work Resolved ([RT states](#)).

State rolls up deterministically from detections to VIs:

- Any Open detection keeps the VI Open.
- A mix of Closed and Stale detections makes the VI **Closed – Fixed**.
- All-Stale detections make it **Closed – Stale**.

At the RT level:

- All-Stale VIs cancel the RT.

- Any false-positive VI blocks auto-close ([VR auto-close setup](#)).

When a VI belongs to several RTs, precedence runs Closed > Deferred > Resolved > In Review > Awaiting Implementation > Under Investigation > Open. An RT closed as Canceled or Fixed with Exception does not close its VIs ([RT/VI states](#)). A VI closed as Fixed, Stale or CI Decommissioned **reopens when a new matching detection arrives**. A Resolved VI that the scanner never confirms as fixed also reopens ([detections and states](#)). A rollup job re-evaluates RT state, reason and Until date every 15 minutes ([RT states](#)).

Stale handling has been rule-based since v22.0. Three base auto-close rules ship with a **90-day** window: Assets last scanned, Detections last found, and Manual detections last found. A daily job moves matching detections to Stale ([auto-close rules](#)). Auto-close acts only on active integration instances and is switched on or off for the whole environment. Tenable needs no special full-run integration for the "Detections last found" rule, unlike Rapid7 and Microsoft TVM, which need a full run in the last seven days ([VR auto-close setup](#)). A separate property closes VIs on retired CIs with the substate CI Decommissioned ([retired CIs](#)). **Exclusion rules** stop matching detections from ever becoming VIs, starting with the next ingestion ([exclusion rules](#)).

Several reference feeds enrich findings before any scanner data arrives, and **NVD (CVE only) and CWE must run before any third-party scanner** ([NVD integration](#)):

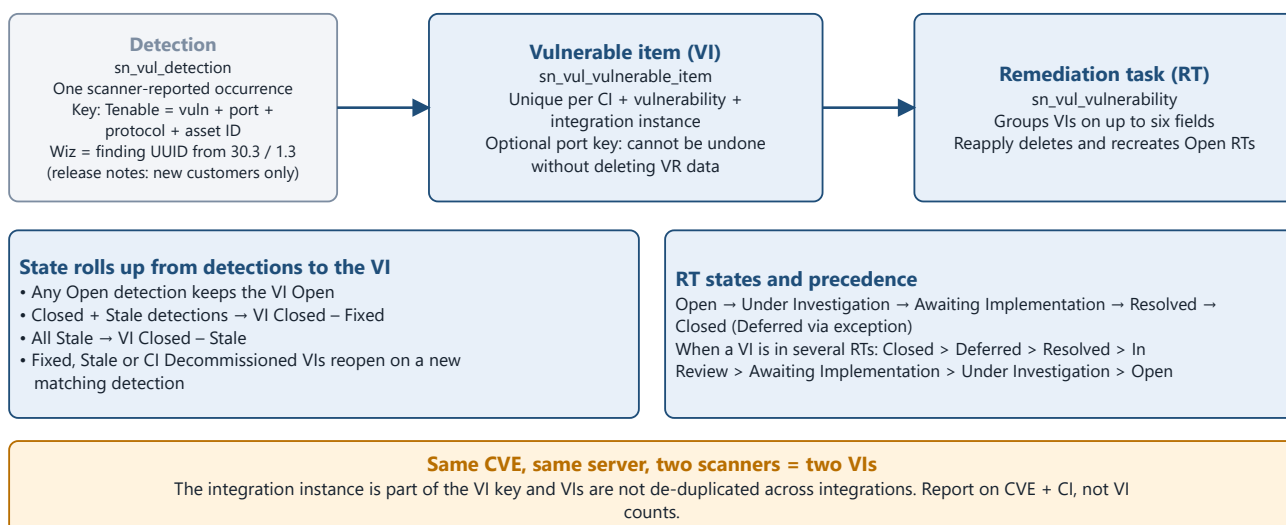
- **CISA KEV** runs daily. It sets CISA Exists, the due date and known-ransomware use, and "the earliest due date is considered for the roll-up to the vulnerable items" ([CISA KEV](#)).
- **FIRST EPSS** runs daily. It writes score and percentile to CVE records and rolls them up to TPEs through a calculator ([EPSS](#)).
- **SSVC** values (Exploitation, Automatable, Technical Impact) are available "only in USEM" ([SSVC](#)).
- **Armis Early Warning** likewise requires VR v30.x ([Armis Early Warning](#)).

Scoring runs through the **Default Risk Calculator**, whose Default Risk Rule weighs severity, exploit information, criticality, external exposure and EPSS. Only one calculator per target field can be active, and **the first matching rule wins**. Ratings band at 90–100 (1), 70–89 (2), 40–69 (3), 1–39 (4) and 0 (5) ([calculators](#)).

Ownership and SLAs are rule-driven:

- **Assignment.** The base rule "Assign to CI support group" stops at the first match. Reapplying assignment rules skips manually assigned VIs ([assignment](#)).
- **Grouping.** RT rules group on up to six fields. A VI joins an existing Open RT with the same assignment group, and **Reapply deletes and recreates Open RTs** ([RT rules](#)).
- **Targets.** The target job applies the most restrictive matching rule, counting from Last opened. USEM 30.0.4 and VR 26.4.4 added four ways to recalculate targets when a risk rating changes ([targets](#)).
- **Exceptions** defer a VI or RT through one- or two-level Flow Designer approval. A first-level approver must exist or nothing can be requested, and an expired exception reverts the item to Open ([exceptions](#)).
- **False positives** close as Closed – False Positive. They are permanent unless the approver sets an Until date ([false positives](#)).
- **Change requests** created from RTs drive the RT state ([change management](#)).
- **Vulnerability Solution Management**, a separate subscription, picks a Preferred solution. Since v24.0.6 it also ingests scanner solutions from the Tenable.io and Tenable.sc plugin integrations ([solution management](#)).

**Figure 3. VR object model: detection, vulnerable item and remediation task, with state roll-up rules.**



Source: Official docs: vr\_host\_detections, vr-configure-vi-key, vr-rt-states, VI state pages.

## CC imports verdicts it never computes

CC "does not calculate the test results, but imports them as part of a third-party integration." It turns scanner-supplied test groups, tests, authoritative sources, citations and pass/fail results into remediation work ([CC test groups](#)). Version 14.9 renamed "Test Result Group" to Remediation Task and "Policy" to Test group, but the tables kept the old names ( `sn_vulc_policy` , `sn_vulc_result_group` ) ([CC overview](#)). Each test carries:

- a scanner Criticality (Critical through Minor);
- a remediation text;
- citations to authoritative sources such as an NIST 800-53 control;
- a **GRC Policy Statements** tab, if GRC Policy and Compliance Management is installed. This tab is the only concrete CC-to-GRC link the docs describe ([CC tests](#)).

Results are Passed, Failed, Error or Unknown. They carry Expected and Actual values, and since v15.0 a passed result keeps its risk score so that mitigated risk can be measured ([CC test results](#)). "Passed items are always in the Closed-Fixed state." After every import an end-of-import event runs three steps:

1. It moves Resolved RTs that still contain failures back to Awaiting Implementation.
2. It closes RTs whose results all passed.
3. It refreshes the "Ungrouped Test Results" list ([CC correlation](#)).

CC's out-of-box automation is thinner than VR's. **The base RT rule ("Assignment group, Test") is disabled by default** ([CC remediation tasks](#)). So are **the base assignment rule** and the "Reapply all assignment rules" job ([CC assignment rules](#)). A fresh CC deployment therefore leaves results ungrouped and unassigned until someone designs the rules. Zurich added a Match First execution mode so that each result lands in exactly one RT ([CC release notes](#)).

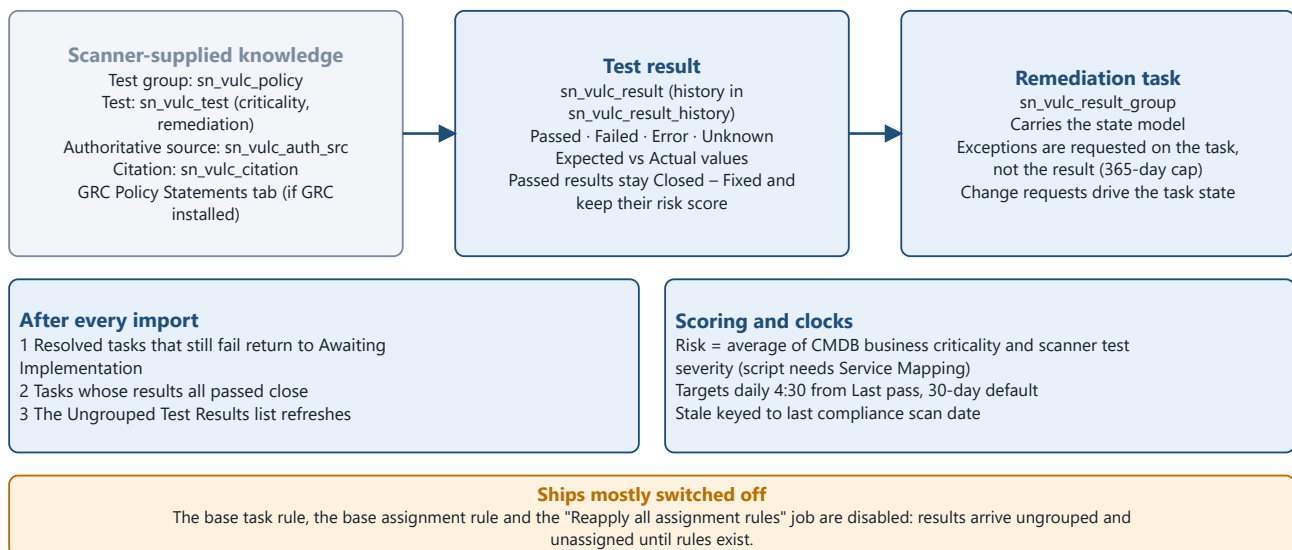
The RT carries the state model. From Resolved, the next scan decides the outcome: all results pass and the RT closes, or it returns to Under Investigation. A resolved result whose Last Seen date is later than its resolution date reopens. When a result sits in several RTs, it takes the highest-precedence state ([CC states](#)). Change-request sync is on by default ([CC change sync](#)):

- Creating or linking a change moves the RT to Awaiting Implementation.
- A change in Review moves it to Resolved.
- A cancelled or unsuccessful change sends it back to Under Investigation.

Scoring, SLAs and exceptions differ from VR in ways that matter for audits:

- **Risk** is "the average of the business criticality of the affected asset as defined in the CMDB, and the severity of the test as communicated by the scanner." The documented script **needs the separately licensed Service Mapping plugin** ([CC risk](#)). "Unknown severity is automatically assigned a risk score of 100" ([CC calculators](#)).
- **Remediation targets** run daily at 4:30. Since v14.12 they count from **Last pass**, falling back to Created, with a **30-day default** ([CC targets](#)).
- **Exceptions** are requested on the RT, not on individual results. The RT sits In review, becomes Deferred on approval and returns to Open on expiry ([CC exceptions](#)). Requests are capped at 365 days by default ([CC exception properties](#)).
- **GRC exceptions**. GRC: Policy and Compliance Management can replace VR as the exception backend, raising a policy exception against a chosen control objective ([GRC exception backend](#)).
- **Staleness** keys on the discovered item's **last configuration-compliance scan date**, not the vulnerability scan date. A server scanned weekly for CVEs but monthly for CIS benchmarks therefore ages differently in CC than in VR ([CC auto-close](#)).

**Figure 4. CC object model: scanner-supplied tests, imported results, and remediation tasks.**



Source: Official docs: Configuration Compliance overview, test, result, remediation-task, risk and target pages.

## CVR anchors findings on images, not running containers

CVR's design principle is to "point to source Docker Image from CVITs instead of running containers." It enriches findings with runtime context (hosts, clusters, namespaces, services) and automatically resolves vulnerabilities "reported in older versions ... when new image versions are deployed at runtime" ([CVR landing](#)). The model has five layers ([CVR tables](#)):

- the discovered container image, holding image ID, digest, registry and last scan date;
- image layers and packages, with a package URL since v2.11.3;
- image findings, one observation per image, package, layer and path;
- CVITs;
- container remediation tasks.

The CVIT key is configured per scanner in `sn_vul_container_image_vulnerability_keys`. Its default is **image repository + vulnerability + image**. ECS adds optional cluster and service components, and EKS adds namespace, registry and service. For example, keying on cluster + service across two clusters and four services yields four CVITs per vulnerability ([CVIT key](#)). Each key record draws cluster and service data from one of two sources:

- **Scanner Information**, taken from the scanner payload.
- **Discovery Information**, taken from ServiceNow Discovery. This needs a daily "Populate image relationships" job, and scanner imports must start **at least four hours** after it completes.

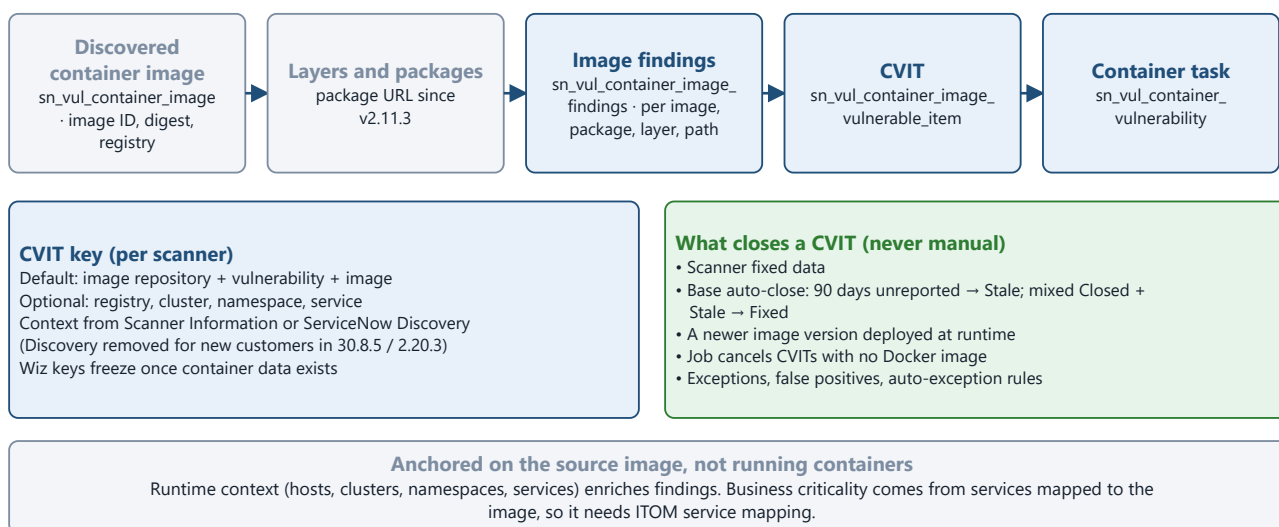
The repo still calls Discovery the default ([key data source](#)). The September 2026 Store release (30.8.5 USEM / 2.20.3) "removed discovery as a data source for determining finding granularity for new customers" ([Store release notes: CVR](#)).

**CVITs cannot be closed manually** ([CVR states](#)). Closure comes from five sources:

- scanner fixed data;
- the base auto-close rule, which moves items unreported for 90 days to Stale and closes CVITs with mixed Closed and Stale findings as Fixed ([CVR auto-close](#));
- image-version supersession;
- a job that cancels CVITs that have no Docker image ([CVR tables](#));
- risk-acceptance paths: multi-level exception and false-positive approvals, plus auto-exception rules that defer matching CVITs ([CVR landing](#)).

Ownership comes from image metadata: repository, labels, cloud account, namespace and cluster ([exploring CVR](#)). RT rules group CVITs on up to six fields drawn from the CVIT, its Docker image or its container vulnerability. **New rules do not touch existing data until Reapply, and deleting a rule can delete its Open tasks** ([CVR RT rules](#)). Risk uses the same bands as VR. Business criticality is computed from the services mapped to the CVIT's Docker image, so it works only where ITOM service mapping links images to services ([CVR calculators](#); [ITOM pattern discovery](#)). Targets count from Last Opened ([CVR dashboard](#)). The Brazil pages document neither a container assignment-rule form nor default container SLA days, so both must be designed from scratch.

**Figure 5. CVR object model, the CVIT key, and every way a CVIT closes.**



Source: Official docs: CVR overview, CVIT key, auto-close and exception pages; Store release notes for 30.8.5 / 2.20.3.

## Separate subscriptions and the v30 fork decide what you can run

The licensing picture is assembled from "separate subscription" statements scattered across product pages. VR is the base entitlement. Each of the following is a separate subscription:

- **CC**, a Store subscription app ([CC](#)).
- **CVR**, "Vulnerability Response and Configuration Compliance for Containers," which requires an entitlement on production instances ([CVR install](#)).
- **The Tenable integration** ([Tenable integration](#)).
- **Vulnerability Solution Management** ([Solution Management](#)) and **Vulnerability Crisis Management** ([Crisis Management](#)).

**The Wiz app lists VR, CVR and CC as prerequisites even for host-only use**, noting that "these applications are available as separate subscriptions" ([Wiz install](#)).

Other features carry hidden dependencies:

- CC's business-criticality risk needs Service Mapping.
- CC dashboards need Performance Analytics for CC ([CC overview](#)).
- The Service Graph Connector for Wiz needs an ITOM Visibility or ITOM Discovery subscription unit, plus a Wiz advanced or standard license ([SGC for Wiz](#)).

The docs publish no pricing or unit definitions. The companion USEM study discusses the Foundation/Advanced/Prime packaging claims, which product documentation does not corroborate.

The **v30 line is the architectural fork**. VR, CC and CVR pages all carry the same instruction: if you do not intend to move to USEM, "install a version below v30.x ... and for upgrades to supported third-party integration applications" ([VR release notes](#); [CC install](#); [CVR install](#)). Scanner apps therefore run on paired version tracks. By September 2026 those tracks stood at:

- Tenable 30.5.3 (USEM) and 6.2.3 (classic);
- Wiz 32.8.4 and 4.2.4;
- CVR 30.8.5 and 2.20.3.

([Store release notes](#): [Tenable](#); [Wiz](#); [CVR](#))

The repo's own version lists lag far behind: the CVR page still lists v30.1/30.2 and v2.1 ([exploring CVR](#)). Compatibility questions all point to the login-gated KB0856498.

Moving to USEM uses the Migration assistant. Install `sn_vul_usem_common` before any v30.x app, rehearse in non-production, upgrade apps in the displayed order, then use **Enable all** to restore integrations and jobs. Only the assistant upgrades the other VR Store apps to v30 and temporarily disables integrations during the cutover ([migrate to USEM](#); [migration planning](#)).

USEM replaces the per-module rule tables with shared ones: `sn_sec_wf_assign_rule`, `sn_sec_wf_ttr_rule`, `sn_sec_rem_task_rule`, `sn_sec_calculator_group`, `sn_vul_cmn_auto_close_rule` and `sn_sec_exception_rule`. It also deprecates CC's `sn_vulc_auto_exception_rule` and `sn_vulc_state_change_approval` ([USEM components](#); [migration prerequisites](#)). One assignment rule can then target VITs, AVITs, CVITs and CC test results through its "Applies to" field ([SEM assignment rules](#)). **The finding tables keep their classic names throughout the Brazil pages**, so reports built on `sn_vul_vulnerable_item`, `sn_vulc_result` or `sn_vul_container_image_vulnerable_item` survive the migration. Rule configuration does not.

Several features exist only on the USEM side: SSVc enrichment, Armis Early Warning, and integration-health dashboards that show queue wait, CI-lookup and rule-processing times ([USEM integration review](#)). Staying classic is therefore a feature freeze, not a neutral choice.

**Figure 6. The v30 fork: classic and USEM tracks must match across every app.**

ServiceNow: if you do not intend to move to USEM, install a version below v30.x of VR, CC, CVR and the scanner integrations.

**Classic track (below v30)**

- VR, CC, CVR classic versions
- CVR 2.20.3 (Sep 2026)
- Wiz 4.2.4 (Sep 2026)
- Tenable 6.2.3 (Sep 2026)
- Per-module rule tables
- No SSVc, Early Warning or integration-health dashboards



**USEM track (v30 and later)**

- Install `sn_vul_usem_common` first
- CVR 30.8.5 (Sep 2026)
- Wiz 32.8.4 (Sep 2026)
- Tenable 30.5.3 (Sep 2026)
- Shared `sn_sec_*` rule tables; one rule can target VITs, AVITs, CVITs and CC results
- Finding tables keep their classic names

Move with the Migration assistant: rehearse in non-production, upgrade apps in the displayed order, then "Enable all". The move cannot be rolled back.

Source: Official docs: VR, CC and CVR install notes, migrate-to-usem; ServiceNow Store release notes (September 2026).

## The integration matrix shows Tenable narrowing and Wiz spanning

| Source                                                           | VR (hosts)                                                                                             | CC (configuration)                                                                                 | CVR (containers)                                                      | Elsewhere                                                      |
|------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|----------------------------------------------------------------|
| <b>Tenable Vulnerability Management</b><br>(formerly Tenable.io) | Assets; Plugin; Fixed → Open Vulnerabilities; Scan Credential; Template; Scan Metadata; rescan         | Fixed Compliance Results → Open Compliance Results; Compliance Results Backfill (v6.1.3 and later) | —                                                                     | Tenable WAS → Application VR                                   |
| <b>Tenable Security Center</b> (Tenable.sc)                      | Open/Fixed Assets; Plugin; Fixed → Open Vulnerabilities; Scan Credential; Backfill; rescan; MID Server | Not documented                                                                                     | —                                                                     | —                                                              |
| <b>Tenable Cloud Security</b><br>(Tenable.cs)                    | Open → Fixed Cloud Host Vulnerabilities                                                                | Not documented                                                                                     | Cloud Container Assets → Open → Fixed Cloud Container Vulnerabilities | —                                                              |
| <b>Wiz</b>                                                       | Host Vulnerability (VMs, serverless); Asset (optional)                                                 | Test Results; Host Test Results; Issues                                                            | Grouped Vulnerability → Deployment Context → Container Vulnerability  | SCA, Secrets, Application List → AVR; AI posture → AI Security |

Sources: [TVM integrations](#); [Tenable.sc integrations](#); [Tenable.cs integrations](#); [Tenable WAS](#); [Wiz integrations by target app](#); [Wiz container chain](#); [Wiz AI posture](#).

Ownership is now clear on both sides. The "Vulnerability Response Integration with Tenable" (scope `sn_vul_tenable`) is built by ServiceNow ([USEM integration catalog](#)). Tenable's own *Tenable and ServiceNow Integration Guide* (revised August 26, 2026) covers only Tenable-built apps:

- the Service Graph Connector for Tenable;
- Tenable for ITSM;
- an OT Exposure app for VR that is "End-of-Life as of September 15, 2025."

The guide lists compatibility through Zurich but not Brazil ([Tenable integration guide](#)). On the Wiz side, a ServiceNow staff announcement on the Community **[Community]** says the ServiceNow-built "Vulnerability Response Integration with Wiz" reached general availability on July 31, 2025. It replaced three Wiz-built apps (Security Operations, Configuration Compliance, Container Vulnerability Response), and new access requests to those apps were no longer granted ([ServiceNow Community announcement](#)). Wiz's public integration page still describes a "Built by Wiz" app and looks stale ([Wiz: ServiceNow VR integration](#)).

# Host findings reach VR through a Fixed-then-Open Tenable chain and a UUID-keyed Wiz feed

---

## Tenable VM and Security Center populate VR in a deliberate order

The Tenable app ingests TVM (cloud), Tenable.sc (on-premises, version 5.13 or later) and Tenable.cs (version 5.0.1 or later) ([Tenable integration](#)). It requires VR 12.1 or later and IntegrationHub. ServiceNow also advises two preparation steps before the first load: **size the instance against the expected VI volume, and disable unused calculators and notification business rules** ([Tenable setup checklist](#)).

For TVM, the integrations run in this order ([TVM integrations](#)):

1. **Assets** creates discovered items and tags. Its export always requests `"is_deleted": false, "is_licensed": true` ([Tenable REST messages](#)).
2. **Plugin** creates TPEs with a TEN- prefix.
3. **Fixed Vulnerabilities** runs on its schedule and then **chains to Open Vulnerabilities**.

Fixed detections update existing VIs but create none, "because Tenable considers Fixed vulnerabilities Mitigated," unless "Create vulnerable items for Fixed Vulnerability detections" is enabled. That option costs performance. Entering TVM credentials activates every TVM vulnerability integration but leaves the compliance integrations off ([Tenable Setup Assistant](#)).

Two defaults shape what arrives:

- **Severity filters import only Critical and High.** Medium, low and info are off ([Tenable retrieval parameters](#)). Medium findings therefore never become VIs and never age into stale closure.
- **The first-run start time** pre-fills three months back, but ServiceNow advises at most one month to avoid Tenable API rate limits and timeouts ([optional modifications](#)).

Advanced JSON filters on the REST methods support `cidr_range`, `plugin_family` and, from app 5.2.1, `epss_score`, `cvss4_base_score` and `vpr_v2_score`. The docs illustrate a split schedule that pulls criticals every four hours and everything else daily ([Tenable filters](#)).

Tenable's API reference adds context the ServiceNow pages leave out ([Tenable: export vulnerabilities](#)):

- The export's `state` filter defaults to OPEN and REOPENED.
- With no time filter, the export covers only items found or fixed in the last 30 days.
- `num_assets` means assets per chunk, ranging from 50 to 5,000.
- `vpr_v2_score` was "scheduled for deprecation on July 1, 2026." A ServiceNow filter built on it should be rechecked.

Tenable.sc uses a different set of integrations ([Tenable.sc integrations](#)):

- Open Assets (cumulative) and Fixed Assets (mitigated).
- Plugin.
- Fixed → Open Vulnerabilities, with plugin families 0 and 39 excluded by default.

- A weekly Scan Credential integration.
- A seven-day Backfill, inactive by default.

Tenable.sc authenticates with API keys from version 5.13. The "Token validation is failed" log line is benign because tokens refresh automatically. A MID Server is mandatory when Tenable.sc and the instance sit in different environments ([Tenable setup checklist](#)). Integrations time out after five minutes whether or not a MID Server is used ([Tenable Setup Assistant](#)). Tenable.cs cloud hosts run in the reverse order, **Open then Fixed**, with `compute_severity_*` filters ([Tenable.cs integrations](#)).

Identity is where Tenable deployments succeed or fail. The base lookup rules differ by product ([Tenable integration](#)):

- **TVM:** MAC address, FQDN, NetBIOS, hostname, DNS, then IP.
- **Tenable.sc:** MAC, FQDN, NetBIOS, then IP.
- **Tenable.cs:** Cloud Resource ID.

"Enable Lookup By Network Partition" adds the TVM `network_id` or the Tenable.sc `repository_id` to the IP lookup rules. IRE then creates distinct CIs for overlapping private address space. A back-fill job, inactive by default, re-stamps existing records ([network partition](#)).

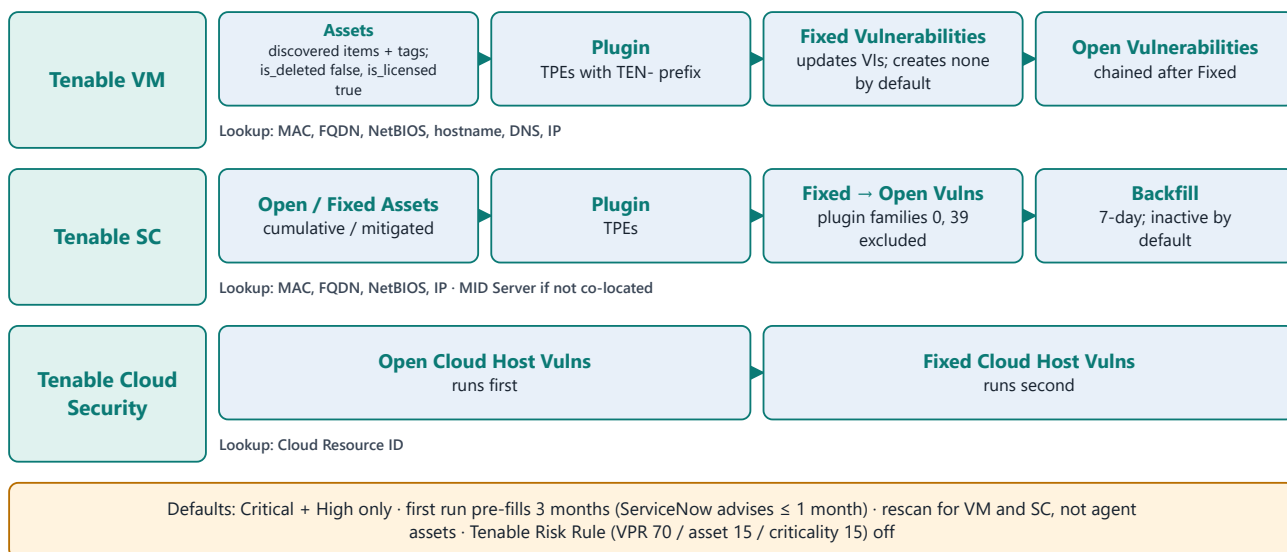
Detections can be split by proof in three steps ([split detections](#)):

1. Set "Include proof in VI key."
2. Enable port granularity.
3. Register plugins with a regex over the plugin output.

The published regex text appears garbled. Asset tags arrive only through the TVM Assets integration, case-insensitively, and the first tag wins. They work in rule conditions but not as an RT group-by key ([Tenable integration](#)).

**Rescan** is available from VI, RT, TPE or discovered-item records for TVM and Tenable.sc. It is not supported on agent-based assets. Child scans are capped at 1,000 IPs, and results arrive with the next Fixed Vulnerabilities import ([TVM rescan](#); [Tenable.sc rescan](#)). The optional **Tenable Risk Rule** weights VPR 70%, asset 15% and business criticality 15%. It is off by default and "may impact your data ingestion performance" ([Tenable Setup Assistant](#)).

**Figure 7. Tenable host paths into VR: job order, defaults and CI lookup rules per Tenable product.**



Source: Official docs: [tenable-io / sc / cs-integrations-list](#), [tenable-data-trieve](#), [tenableIntegration](#); Tenable developer docs.

## Wiz host findings key on the finding UUID and stamp their own assets

The Wiz app (scope `sn_vul_wiz`) authenticates with a Wiz service account: Auth URL, API URL, Client ID and Client Secret, then "Save and test." The Host Vulnerability integration needs the Wiz scopes `read:host_configuration` and `read:vulnerabilities`. All integrations run **daily** by default except Host Test Results, which runs on demand, and "Import since" backdates a run ([Wiz configuration](#)).

From 30.3 (USEM) and 1.3 (classic), **the detection key is the Wiz finding UUID**, replacing vulnerability + asset\_id + proof, and any key change needs a full import ([Wiz configuration](#)). The release notes restrict the UUID key to new customers ([VR release notes](#)).

From 32.1 (USEM) and 4.1 (classic), the separate Asset integration is **off by default and optional**, because the host integration stamps discovered items directly from `vulnerableAsset`. If it is turned on with no resource types selected, it pulls every type. The Host Vulnerability integration needs at least `VIRTUAL_MACHINE` or `SERVERLESS` ([Wiz resource types](#)).

The Wiz filters carry real prioritization signal:

- Has Public Exploit
- Has CISA KEV Exploit
- Has Fix
- Resource Has Wide or Limited Internet Exposure
- admin or high privileges
- detection method
- status (OPEN, REJECTED, RESOLVED)
- **Validated In Runtime**, a Wiz status that "typically persists for a 48-hour period" after the package was last seen in memory

A filter left at `--None--` means "no data is imported for this field" ([Wiz host filters](#)).

Because Wiz feeds both VR and Application VR, lookup rules should set **Applies to** (Discovered Item or Discovered Application). Leaving it blank lets the two reapply jobs fight over the same flag ([lookup rules](#)).

Two gaps matter. **The docs never map Wiz host status to VI state.** REJECTED lands only on detection status and `is_ignored`. By contrast, Application VR documents that rejected findings become Deferred/Risk Accepted unless ServiceNow exception management is selected ([Wiz AVR configuration](#)). And **no rescan exists for Wiz:** the USEM catalog lists rescan only for Tenable and Qualys ([USEM integration catalog](#)).

| VR target                                 | Tenable source (TVM unless noted)                                                                              | Wiz source (Host Vulnerability / Asset)                                                                                                                              |
|-------------------------------------------|----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Discovered item<br><code>source_id</code> | Asset <code>id</code> (UUID)                                                                                   | <code>vulnerableAsset.id</code> (asset integration: <code>id</code> )                                                                                                |
| Host identifiers                          | First <code>ipv4s</code> , <code>fqdns</code> , <code>netbios_names</code> ,<br><code>mac_addresses</code>     | <code>vulnerableAsset.name</code> → <code>dns</code> ; <code>ipAddresses[0]</code> →<br><code>ip_address</code>                                                      |
| Cloud context                             | Cloud metadata fields (v18+)                                                                                   | <code>region</code> , <code>providerUniqueId</code> , <code>cloudPlatform</code> ,<br><code>subscriptionExternalId</code> , <code>nativeType</code>                  |
| Internet exposure                         | —                                                                                                              | <code>isAccessibleFromInternet</code> →<br><code>cmdb_ci_internet_facing</code> (Asset)                                                                              |
| TPE ID                                    | Plugin <code>id</code> → <code>TEN-&lt;id&gt;</code>                                                           | <code>name</code> → <code>sn_vul_entry.id</code>                                                                                                                     |
| Severity                                  | <code>risk_factor</code> → <code>source_severity</code> ;<br><code>severity_id</code> → VI priority            | <code>vendorSeverity</code> → <code>source_severity</code>                                                                                                           |
| Vendor risk                               | <code>vpr.score</code> (v2 preferred) →<br><code>source_risk_score</code> + rating                             | <code>score</code> → <code>v3_base_score</code> (no Wiz risk score)                                                                                                  |
| KEV / exploit                             | <code>on_cisa_kev</code> → <code>cisa_exists</code> ;<br><code>exploit_available</code> → <code>exploit</code> | <code>hasCisaKevExploit</code> → <code>cisa_exists</code> ; <code>hasExploit</code> →<br><code>exploit</code>                                                        |
| EPSS                                      | <code>epss_score</code> → Tenable TPE Additional<br>Attributes                                                 | From the CVE-level EPSS feed only                                                                                                                                    |
| Proof / fix                               | <code>sc_plugin_text</code> → <code>proof</code> ; <code>solution</code><br>→ TPE                              | <code>description</code> → <code>proof</code> ; <code>remediation</code> →<br><code>solution_summary</code> ; <code>fixedVersion</code> → <code>fixed_version</code> |
| State                                     | <code>state</code> → VI state ( <code>sc</code><br><code>hasBeenMitigated</code> )                             | <code>status</code> → detection <code>status</code> , <code>source_status</code> ,<br><code>is_ignored</code>                                                        |

Sources: [Tenable transform reference](#); [Wiz field mappings](#). Tenable and Wiz both write KEV to the same `cisa_exists` field that the CVE-level CISA feed rolls up to TPEs. **Risk rules should reference one canonical KEV and EPSS signal to avoid counting the same fact twice.**

# Configuration verdicts reach CC only from Tenable VM and Wiz

---

## Tenable VM compliance arrives as Fixed and Open buckets plus a backfill

**Only TVM feeds CC.** The CC integrations page names only "the Tenable.io product of the Tenable Vulnerability Integration" ([CC integrations](#)), and every Tenable.sc integration is vulnerability-only ([Tenable.sc integrations](#)). Security Center estates therefore have no documented CC path. The Setup Assistant page requires CC v12.2 or later ([Tenable Setup Assistant](#)).

App v6.1.3 removed the single Compliance Results integration and split it in two ([TVM integrations](#)):

- **Fixed Compliance Results** imports PASSED and SKIPPED results. It is scheduled and chains to the next integration.
- **Open Compliance Results** imports FAILED, WARNING, ERROR and UNKNOWN results. It does not run if Fixed is inactive or fails.

A result whose asset cannot be matched is **ignored**. Its asset ID is parked in `sn_vul_tenable_missing_asset`, and **Compliance Results Backfill** reconciles up to 200 of those IDs per run after the Assets integration. In ServiceNow's worked example, 20 of 100 assets are ignored, and reconciliation "may take multiple runs" ([CC Tenable overview](#)). The 200 figure matches Tenable's own cap of 200 asset UUIDs per compliance-export request ([Tenable: export compliance data](#)).

On the vendor side, compliance data exists only if TVM scans run audit files: Tenable-supplied CIS or DISA STIG audits, or custom `.audit` files, each needing appropriate credentials ([Tenable VM: compliance in scans](#)). Tenable's compliance export aggregates results across scans rather than reporting them per scan ([Tenable: export compliance data](#)). It now works with Basic [16] permissions plus Can View on the assets ([Tenable changelog](#)).

**Test identity is the decision that matters most.** Tenable compliance tests used to be keyed on `check_id`. When several tests shared it, "later ingestion runs overwrite earlier records, causing data loss." Three keys are now available ([test uniqueness key](#)):

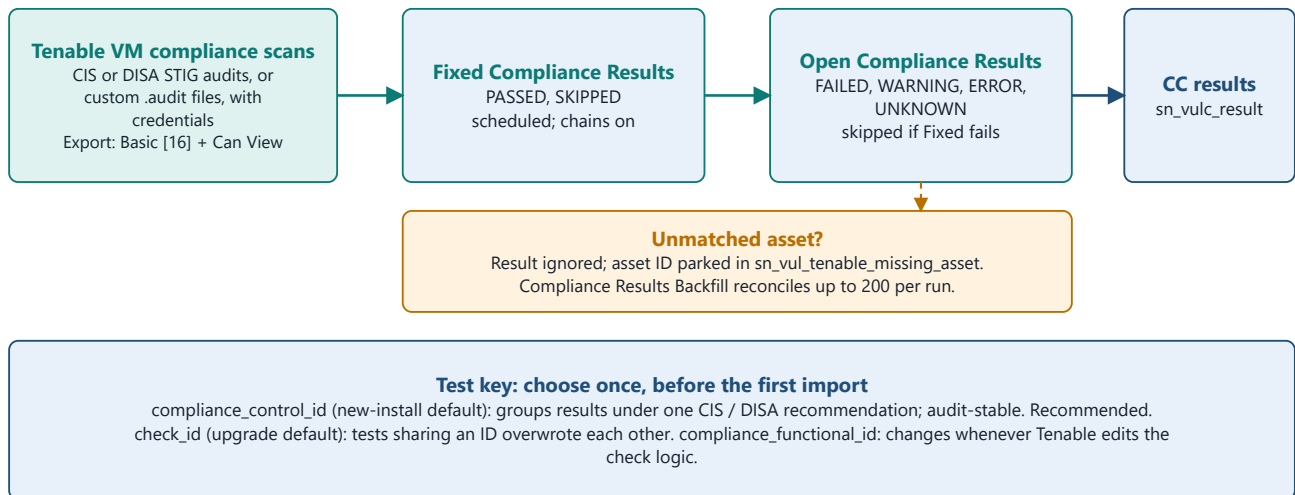
- `compliance_control_id`, the default for new installs;
- `check_id`, kept as the default on upgraded instances;
- `compliance_functional_id`.

Only one key can be active, and changing it changes how later runs match records. It is set under "Configure Tenable Test Granularity" ([set the key](#)).

Tenable defines `compliance_control_id` as a computed hash that groups results under one CIS or DISA recommendation. It defines `compliance_functional_id` as a hash of the audit code that **changes whenever the check's evaluation logic changes** ([Tenable changelog](#)). The control ID is therefore the audit-stable choice. The functional ID would create a new CC test every time Tenable edits an audit.

Since v15.6.1, result granularity can add keys such as `instance`, so a database with five instances yields five results. ServiceNow advises no more than three extra keys because each adds run time ([result granularity](#)). Tenable supplies **no risk score** for compliance results. The transform reference says ServiceNow applies "the default Medium risk value of 20" in that case ([Tenable transform reference](#)), but two other pages disagree (see the contradictions table).

**Figure 8. Tenable compliance path into CC, the backfill, and the test-key decision.**



Source: Official docs: [tenable-io-integrations-list](#), [Tenable test granularity page](#), [CC Tenable pages](#); [Tenable compliance export docs](#).

## Wiz writes three kinds of CC results and can divert AI posture for good

Three Wiz integrations write CC results ([Wiz integrations by target app](#)):

- **Test Results:** non-compliant cloud configurations.
- **Host Test Results:** VIRTUAL\_MACHINE host configuration. These arrive from integration v1.1 with result type `host_misconfiguration`.
- **Issues:** CLOUD\_CONFIGURATION, THREAT\_DETECTION and TOXIC\_COMBINATION, labeled "Wiz Issues."

These integrations need the `read:issues` and `read:threat_issues` scopes ([Wiz configuration](#)). Version 1.1 made four changes ([CC release notes](#)):

- It deprecated the missing-asset backfill; primary integrations must be **backdated three days and re-run**.
- It replaced `is_ignored` with `is_result_ignored`.
- It mapped Wiz source severity to Priority on `sn_vulc_result`.
- It populated `validated_at_runtime`.

The Test Results tab filters on severity, project, cloud platform (including GitHub, Terraform and OpenAI), subscription, Framework Category, resource and native type, Has remediation, and status. Two options govern rejected findings: **Fetch rejected** imports Wiz-rejected results that "remain in a failed state but are not rolled up," and **Close rejected** imports them and closes them ([Wiz Test Results filters](#)). The Issues tab adds an Issue

type filter ([Wiz Issues filters](#)). That filter deserves a deliberate setting: by default, runtime THREAT\_DETECTION issues would land in CC as configuration results, which is a modeling compromise. Many teams would rather route them to incident response.

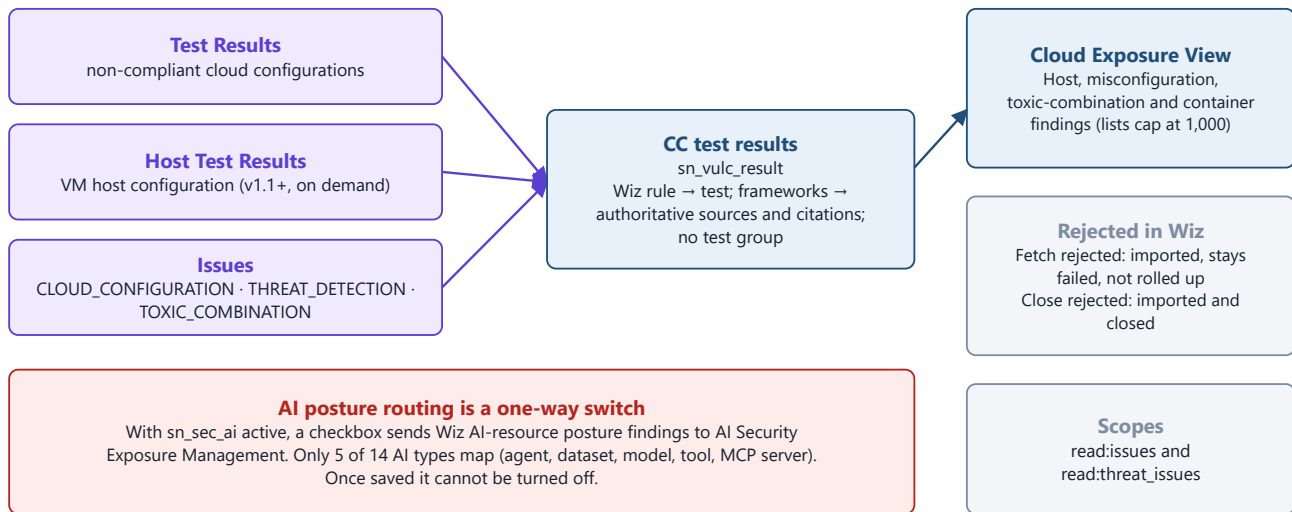
**AI posture routing is a one-way switch.** With the `sn_sec_ai` plugin active, a checkbox on the Test Results tab sends Wiz AI-resource posture findings to AI Security Exposure Management. Wiz defines 14 AI resource types, but **only five map to an AI Security asset type**: agent, dataset, model, tool and MCP server. Evidence is not populated ([Wiz AI-SEM integration; mapping](#)). **Once saved, the setting cannot be turned off**, even through the API, and CC "stops creating test results" for routed findings ([Wiz AI-SEM configuration](#)).

The **Cloud Exposure View** groups Wiz-sourced data into Host, Misconfiguration, Toxic combination and Container findings. Its lists cap at 1,000 records ([Cloud Exposure View](#)). Matching Wiz CC results to CMDB cloud CIs works only if the lookup rules reproduce Discovery's `object_id` formats, for example `arn:aws:s3:::<bucket>` for S3. A mismatched format silently misses ([cloud CI lookup](#)).

| CC target                                   | Tenable VM compliance source                                                          | Wiz Test Results source (Issues variant)                                                                     |
|---------------------------------------------|---------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| Test group<br><code>sn_vulc_policy</code>   | <code>audit_file</code> → short description                                           | None documented                                                                                              |
| Test<br><code>sn_vulc_test.source_id</code> | <code>check_id</code> , or the configured key                                         | <code>rule.id</code> ( <code>sourceRule.id</code> )                                                          |
| Test name / remediation                     | <code>check_name</code> ; <code>check_info</code> ; <code>solution</code>             | <code>rule.name</code> ; <code>rule.remediationInstructions</code> ( <code>resolutionRecommendation</code> ) |
| Test criticality                            | None. Tenable sends no risk score                                                     | <code>severity</code> → <code>source_criticality</code> ; source severity → result Priority (v1.1)           |
| Result value                                | <code>status</code> → <code>result</code>                                             | <code>status</code> → <code>result</code> ; Rejected → ignored flag                                          |
| Expected / actual                           | <code>expected_value</code> / <code>actual_value</code>                               | Not mapped                                                                                                   |
| Authoritative source                        | <code>reference.framework</code>                                                      | <code>securitySubCategories.category.framework.name</code>                                                   |
| Citation                                    | <code>reference.control</code> → section;<br><code>profile_name</code> → section name | <code>securitySubCategories.id</code> / <code>.title</code>                                                  |
| Dates                                       | <code>first_seen</code> / <code>last_seen</code>                                      | <code>firstSeenAt</code> / <code>analyzedAt</code> ( <code>createdAt</code> / <code>updatedAt</code> )       |
| Technology                                  | <code>db_type</code>                                                                  | Not mapped                                                                                                   |
| Asset                                       | <code>asset_uuid</code> → discovered item                                             | <code>resource.*</code> ( <code>entitySnapshot.*</code> ) → discovered item <code>source_data</code>         |

Sources: [Tenable transform reference](#); [Wiz field mappings](#). No Wiz field is documented as populating a test group. Wiz results will likely have no test group unless one is created locally, and **CC reporting by benchmark should pivot on authoritative source and citation, not test group**.

**Figure 9. Wiz paths into CC, rejected-finding options, and the one-way AI posture switch.**



Source: Official docs: [wiz-test-result-tab-filters](#), [wiz-issues-tab-filters](#), [wiz-ai-sem-integration](#), [vr-cloud-exposure-view-db](#).

## Container data reaches CVR through Tenable.cs or a three-step Wiz chain

### Tenable.cs sends an explicit fixed signal

Tenable container data comes **only from Tenable Cloud Security**, through three chained integrations ([Tenable.cs integrations](#)):

1. **Cloud Container Assets** creates discovered container images, Docker images and repositories.
2. **Open Cloud Container Vulnerabilities** creates new or reopened CVITs, findings, TPEs and CVEs.
3. **Fixed Cloud Container Vulnerabilities** creates findings "in closed state."

Authentication is a Tenable.cs API token, and the container forms in Setup Assistant appear "only if Container Vulnerability Response is installed" ([Tenable Setup Assistant](#)). Transport is a GraphQL REST message ([Tenable REST messages](#)). Container severity filters again default to Critical and High only ([Tenable retrieval parameters](#)).

The image digest ( `u_digest` ) becomes both `image_id` and `image_digest` and "is used for CI lookup." `u_resolved` drives the finding status and the CVIT state, and VPR lands in `source_risk_score` ([Tenable transform reference](#)). Because a fixed signal arrives explicitly, Tenable.cs closure is more deterministic than reliance on the 90-day stale rule.

Two cautions apply. The Store notes put real Tenable.cs ingestion at app 5.0.3 (May 2025) and record a fix in 30.3.3/6.1.3 for "a backward-incompatible API change" on Tenable's side ([Store release notes: Tenable](#)). And Tenable's legacy token page now carries an end-of-support notice, which leaves the token's permission model

undocumented ([Tenable Cloud Security: API tokens](#)). The 70/15/15 Tenable Risk Rule is documented only for host VIs; CVR has no equivalent.

## Wiz now maps every deployment before it creates CVITs

From Store release 32.8.4 (USEM) and 4.2.4 (September 2026), Wiz container data runs as a chain that is active by default ([Wiz container chain](#)):

1. **Container Grouped Vulnerability** gathers Source IDs for active images into `sn_vul_container_image`.
2. **Container Deployment Context** maps each image to cluster, namespace and service in `sn_vul_container_image_relationship_mapping`, and creates missing CVITs for new deployments.
3. **Container Vulnerability** creates CVITs for every cluster, namespace or service the image is deployed on.

Run alone, the Container Vulnerability integration created CVITs on **at most 16 clusters** per image. The Store notes describe this as a "16-controller-per-image cap" that also made auto-closing unreliable when a deployment was removed ([Store release notes: Wiz](#)). Setting `sn_vul_wiz.deployment_context_gate_writes` to false turns the chain off and restores the old behavior. That makes sense only if clusters are not part of the CVIT key.

Several earlier changes reshaped Wiz container identity. In 32.0.3, finding uniqueness gained the **path** attribute. Existing findings were migrated, and irrelevant ones were **bulk-closed as invalid**, which a metrics team must not read as remediation. The same release rolls Validated in Runtime up from findings to CVITs ([Store release notes: Wiz](#)). Zurich-era releases ([CVR release notes](#)):

- changed the repository name to the form registry/repository, with a separate finding per registry;
- rolled fix status up to CVITs as Fix available, Partial fix available or No fix available;
- evaluated cluster and namespace for DEPLOYMENT, DAEMON\_SET, STATEFUL\_SET and POD controllers.

Wiz CVIT keys "aren't editable unless there is no container vulnerability data in the instance, or unless you're a new customer" ([Wiz container chain](#)). The container import filters include Has Fix, Has CISA KEV Exploit, Limited Internet Exposure, Detection Method and Validated In Runtime ([Wiz container filters](#)). CVR 30.8.5/2.20.3 adds ingestion of Wiz Running Container Vulnerability findings ([Store release notes: CVR](#)).

CMDB alignment runs through `cmdb_ci_docker_image`. The AWS Service Graph Connector models the image as instantiating Docker containers, with Kubernetes clusters containing namespaces, pods and services ([AWS CMDB classes](#)). If those classes are not populated with matching image IDs, choose Scanner Information as the key data source.

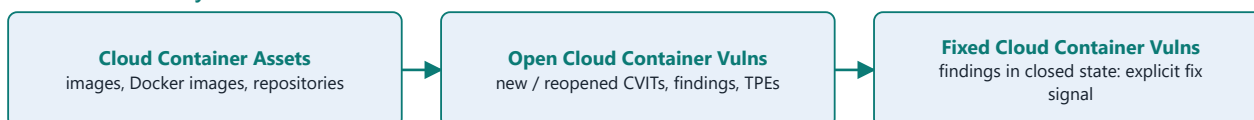
| CVR target            | Tenable.cs source                                                                     | Wiz source                                                                           |
|-----------------------|---------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Image ID / digest     | <code>u_digest</code> → <code>image_id</code> , <code>image_digest</code> (CI lookup) | <code>imageId</code> → <code>image_id</code>                                         |
| Image name            | <code>u_name</code>                                                                   | <code>vulnerableAsset.name</code>                                                    |
| Repository / registry | <code>u_repositoryuri</code>                                                          | <code>repository.externalId</code> : before <code>##</code> → repo, after → registry |
| Labels / tags         | <code>u_labels</code> ; <code>u_imagetags</code>                                      | <code>tags</code> → <code>image_labels</code>                                        |

| CVR target             | Tenable.cs source                                                                                                   | Wiz source                                                                                                        |
|------------------------|---------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Cluster / namespace    | <code>u_clusters</code> → <code>image_cluster</code>                                                                | <code>executionControllers.ancestors.name</code> → <code>image_namespace</code> , <code>image_clusters</code>     |
| Cloud context          | <code>u_cloudprovider</code> , <code>u_accountid</code> , <code>u_region</code>                                     | <code>cloudPlatform</code> , <code>subscriptionExternalId</code> , <code>region</code>                            |
| Package                | Software name, version, type, paths                                                                                 | <code>detailedName</code> , <code>version</code> , <code>locationPath</code>                                      |
| Vendor risk / severity | <code>VprScore</code> → <code>source_risk_score</code> ;<br><code>VprSeverity</code> → <code>source_severity</code> | Vendor severity → finding source severity                                                                         |
| Status                 | <code>u_resolved</code> → finding status and CVIT state                                                             | <code>status</code> → finding <code>is_ignored</code>                                                             |
| Runtime / base layer   | —                                                                                                                   | <code>validate_at_runtime</code> ;<br><code>layerMetadata.isBaseLayer</code> → finding <code>is_base_image</code> |
| Fix                    | —                                                                                                                   | <code>fixed_version</code> → finding <code>fix_status</code>                                                      |

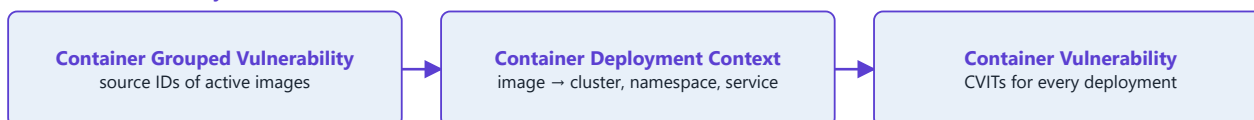
Sources: [Tenable transform reference](#); [Wiz field mappings](#). Because CVR 30.3.2 deprecated the CVIT-level `IsBaseImage` flag as "image-specific data" ([Store release notes: CVR](#)), triage of base-image versus application-layer findings should filter on findings or layers, not CVITs.

**Figure 10. Container paths into CVR: the Tenable Cloud Security chain and the Wiz deployment-context chain.**

#### Tenable Cloud Security



#### Wiz (32.8.4 / 4.2.4, on by default)



#### Tenable.cs specifics

- Image digest → `image_id` and `image_digest`, used for CI lookup
- `u_resolved` drives finding status and CVIT state
- Critical + High only by default
- Token permissions undocumented (vendor page end-of-support)

#### Wiz specifics

- Old Container Vulnerability alone capped CVITs at 16 clusters per image
- 32.0.3 added path to uniqueness; irrelevant findings bulk-closed as invalid
- Fix status and Validated in Runtime roll up to CVITs
- Gate: `sn_vul_wiz.deployment_context_gate_writes`

Source: Official docs: [tenable-cs-integrations-list](#), [wiz-container-runtime-exposure-cvr](#), [wiz-container-tab-filters](#); [Store release notes](#).

## Least-privilege setup starts on the vendor side

| Requirement       | Tenable VM                                                                              | Tenable Security Center                                   | Tenable Cloud Security                         | Wiz                                                                                                                                                                |
|-------------------|-----------------------------------------------------------------------------------------|-----------------------------------------------------------|------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ServiceNow apps   | VR 12.1+ with NVD and CWE loaded; IntegrationHub; Tenable app; CC 12.2+ for compliance  | Same, plus a MID Server unless co-located                 | Same, plus the CVR app for containers          | VR with NVD, CISA and CWE; CVR app; CC app; Wiz app                                                                                                                |
| ServiceNow roles  | admin (install);<br>sn_vul.vulnerability_admin;<br>sn_vul_tenable.configure_integration | Same                                                      | Same                                           | sn_vul.vulnerability_admin;<br>sn_vulc.admin;<br>sn_vul_wiz.configure_integration;<br>sn_vul_wiz.read_integration                                                  |
| Vendor credential | API access and secret keys                                                              | API keys (5.13+, after an admin enables "Allow API Keys") | API token                                      | Service account: Client ID, Client Secret, Auth URL, API URL                                                                                                       |
| Vendor permission | Basic [16] from app v3.8 (Administrator [64] before); Can View on assets for exports    | Security Analyst or Security Manager                      | Not documented (vendor page is end-of-support) | Host: read:vulnerabilities, read:host_configuration . Asset: read:resources . Container: read:cloud_configuration . CC and Issues: read:issues, read:threat_issues |
| Throughput limits | 10 concurrent exports per container; 429 with retry-after; num_assets 50–5,000          | 30-second sync calls; 5-minute integration timeout        | Asset page ≤1,000; vulnerability page ≤10,000  | First page size (asset default 500; 500–1,000 suggested for findings); Wiz-side limits not public                                                                  |

Sources: [Tenable setup checklist](#); [Tenable Setup Assistant](#); [Tenable retrieval parameters](#); [Tenable: concurrency limiting](#); [Tenable: roles](#); [Tenable SC: API key authentication](#); [Wiz install](#); [Wiz configuration](#); [Wiz container filters](#).

**On the Tenable side**, the numeric TVM roles are Read-Only 0, Basic 16, Scan Operator 24, Standard 32, Scan Manager 40 and Administrator 64. Custom roles can carry granular export privileges ([Tenable: roles](#)). A Basic service user with Can View on the relevant asset objects is therefore the least-privilege TVM identity for ServiceNow from app v3.8. Rescan is the exception: it relies on the Scan Credential and Template integrations, and the docs do not state what scan permissions it needs.

Tenable.sc API keys require an administrator to enable "Allow API Keys" first. The secret cannot be viewed again after generation, and regenerating keys de-authorizes the old ones ([Tenable SC: manage API keys](#)).

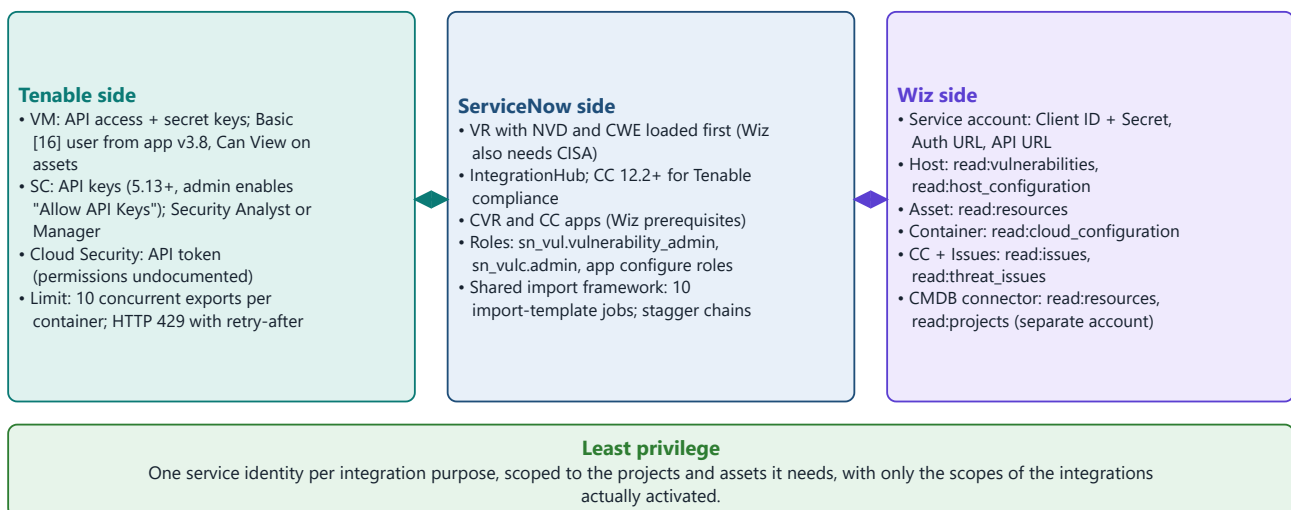
Tenable caps export endpoints at **10 concurrent exports per container**, rejects duplicate exports with the same filters, and returns HTTP 429 with a `retry-after` header ([Tenable: concurrency limiting](#)). That cap is shared by every tool pulling from the same TVM container: ServiceNow's asset, fixed, open and compliance chains, plus any data lake or SIEM exporter.

Do not confuse the Tenable-built apps with the ServiceNow-built one. They use their own `x_tsi*rm_*` roles, cross-scope privileges and chunk defaults ([Tenable integration guide](#)). Tenable's "Create ServiceNow Ticket" action writes incidents directly, bypassing VR. It needs a Tenable One license and is limited to 200 findings per action ([Tenable VM: create a ticket](#)).

**On the Wiz side**, creating a service account requires a Wiz user with Write permission on service accounts. Project-scoped roles can create accounts only within their own projects, and you can select projects to limit what the account sees ([SGC for Wiz setup](#)). The token URL is `https://auth.app.wiz.io/oauth/token`, and API URLs take the form `https://api.<region>.app.wiz.io` ([SGC for Wiz configuration](#)). The CMDB connector needs only `read:resources` and `read:projects`. That makes a separate CMDB service account, distinct from the security-findings account, the natural least-privilege split. Government-cloud endpoints, GraphQL rate limits and Wiz-side ticketing could not be verified because Wiz's documentation requires a login.

**On the ServiceNow side**, every scanner shares one import framework. By default it has **ten "Vulnerability Import Template" engine jobs**, one-hour import-queue entries kept alive by heartbeats, and a 60-minute run-timeout checker. Chained integrations trigger their successors ([VR components](#); [VR integrations](#)). Large Tenable and Wiz estates therefore compete for the same workers, so stagger the chains. Domain-separated Tenable imports need a run-as user per domain and a cloned data source processor ([optional modifications](#)). CVITs ingest into the integration user's domain ([CVR domain separation](#)).

**Figure 11. Least-privilege setup across Tenable, ServiceNow and Wiz.**



Source: Official docs: [tenable-setup-checklist](#), [vr-wiz-host-vuln-install](#), [vr-config-wiz-host-vuln](#), [sgc-cmdb-wiz-setup](#); Tenable developer docs.

## Two dozen documentation contradictions touch design decisions

The Brazil pages carry a September 2026 date, but many paragraphs predate the code they describe. These conflicts change how a team would configure the product:

| Topic                           | Conflict                                                                                                                                                                                                                 | Practical resolution                                                                                     |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Tenable app ownership           | <a href="#">VR integrations</a> says the Tenable app is "created and maintained by Tenable"; <a href="#">USEM catalog</a> says "Built by ServiceNow"                                                                     | Treat <code>sn_vul_tenable</code> as ServiceNow-built; migration from the Tenable-built app is KB0960667 |
| Tenable.sc auth                 | <a href="#">Setup Assistant</a> lists user/password and API key, then says "only API key" is supported                                                                                                                   | Use API keys (Tenable.sc 5.13+)                                                                          |
| Tenable severity mapping        | <a href="#">Severity mapping</a> maps Priority to <code>source_severity</code> and VPR in the Plugin run; <a href="#">transform reference</a> maps severity to <code>priority</code> and VPR in the vulnerability import | Inspect the transform maps on the instance                                                               |
| <code>num_assets</code> meaning | ServiceNow: "max vulnerabilities per exported chunk" ( <a href="#">retrieval parameters</a> ); Tenable: assets per chunk, 50–5,000 ( <a href="#">Tenable API</a> )                                                       | Follow Tenable's definition when tuning                                                                  |
| Tenable CC integration name     | <a href="#">CC Tenable overview</a> still says run "Compliance Results Integration"; <a href="#">TVM list</a> says it was removed in v6.1.3                                                                              | Activate both Fixed and Open Compliance Results                                                          |
| Tenable CC test key             | Transform hard-codes <code>check_id</code> → <code>source_id</code> ; <a href="#">uniqueness key page</a> makes it configurable                                                                                          | Choose <code>compliance_control_id</code> before first import                                            |
| Tenable CC risk                 | "Default Medium risk value of 20" ( <a href="#">transform reference</a> ) vs vendor scores in Criticality ( <a href="#">CC risk example</a> ) vs unknown = 100 ( <a href="#">CC calculators</a> )                        | Test a known result's score; set an explicit calculator rule                                             |
| CC auto-close roll-up           | All Closed-Fixed → RT Closed-Canceled ( <a href="#">procedure</a> ) vs all Closed-Stale → Closed-Canceled ( <a href="#">overview</a> )                                                                                   | The overview matches VR's logic; verify on the instance                                                  |
| CC RT creation and Close        | RTs "are created manually" ( <a href="#">correlation</a> ) vs RT rules ( <a href="#">RT rules</a> ); the state page still cites the Close button removed in v15.0                                                        | RT rules exist but ship disabled                                                                         |
| CC CI lookup scope              | "Only for the Qualys Integration," then lists Defender and Prisma rules ( <a href="#">CC CI rules</a> )                                                                                                                  | Test lookup coverage per scanner                                                                         |
| Wiz host configurations         | "Host configurations from Wiz are not imported" ( <a href="#">Test Results filters</a> ) vs the Host Test Results integration from v1.1 ( <a href="#">Wiz CC overview</a> )                                              | Host configuration is imported from v1.1                                                                 |
| Wiz Asset prerequisite          | Same page says "optional ... deactivated by default" and "other integrations depend on" it ( <a href="#">resource types</a> )                                                                                            | Optional from 32.1/4.1; enable it only for exposure flags                                                |
| Wiz detection key               | UUID from 30.3/1.3 ( <a href="#">configuration</a> ) vs "new customers only" ( <a href="#">VR release notes</a> ); the configure page contains unedited draft questions                                                  | Check the Detection Granularity record before upgrading                                                  |
| Wiz ignored flag                | <code>is_ignored</code> in <a href="#">field mappings</a> vs <code>is_result_ignored</code> in <a href="#">CC release notes</a>                                                                                          | Build reports on <code>is_result_ignored</code> for CC                                                   |
| Wiz AI routing                  | "Instead of Configuration Compliance" ( <a href="#">configure</a> ) vs "into AI-SEM and Configuration Compliance" ( <a href="#">overview</a> )                                                                           | Assume CC stops; the setting is irreversible                                                             |
| Wiz in CC catalog               | Missing from <a href="#">CC integrations</a> ; the <a href="#">USEM catalog</a> Wiz CC entry says "from Microsoft Defender for Cloud"                                                                                    | Copy errors; Wiz CC is supported                                                                         |
| Default CVIT key                | Repository + tag + vulnerability ( <a href="#">exploring CVR</a> ) vs repository + vulnerability + image ( <a href="#">key granularity</a> )                                                                             | Read <code>sn_vul_container_image_vulnerability_keys</code>                                              |

| Topic                               | Conflict                                                                                                                                                                                         | Practical resolution                                                                         |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| CVR Discovery default               | Discovery is the default ( <a href="#">key data source</a> ) vs removed for new customers in 30.8.5/2.20.3 ( <a href="#">Store notes</a> )                                                       | Use Scanner Information unless Kubernetes Discovery is mature                                |
| Base-image tracking                 | CVIT "Base Image" field ( <a href="#">CVIT fields</a> ) vs CVIT IsBaseImage deprecated ( <a href="#">Store notes</a> )                                                                           | Filter on finding <code>is_base_image</code>                                                 |
| Wiz 16-item cap; container path     | "16 clusters" ( <a href="#">chain page</a> ) vs "16-controller" ( <a href="#">Store notes</a> ); path moved to findings ( <a href="#">Australia notes</a> ) but mappings still target packages   | Keep the chain on; read path from findings                                                   |
| Wiz container scopes                | Container integration lists only<br><code>read:cloud_configuration</code> ( <a href="#">configuration</a> )                                                                                      | Test the service account with a dry run; add <code>read:vulnerabilities</code> if calls fail |
| Tenable.cs Fixed output and version | Fixed integration output listed as "New/Reopened" CVITs ( <a href="#">Tenable.cs list</a> ); repo says Tenable.cs from 5.0.1, Store notes say ingestion from 5.0.3                               | Fixed closes findings; run 5.0.3 or later                                                    |
| VR remediation-target defaults      | Shipped rules (15/30/45 days, inactive) "applicable only for application vulnerable items" ( <a href="#">SEM targets</a> ) vs "at least one rule is shipped" ( <a href="#">Setup Assistant</a> ) | Author host SLA rules explicitly                                                             |
| Enrichment order                    | NVD "runs daily" ( <a href="#">NVD</a> ) vs "weekly on Monday" ( <a href="#">CWE jobs</a> ); EPSS before vs after scanner imports ( <a href="#">EPSS</a> )                                       | Schedule NVD → CWE → KEV → EPSS ahead of scanner chains, daily                               |

Two more process-level conflicts belong on the risk register. The Setup Assistant still tells admins to assign the deprecated `sn_vul.admin` role ([Setup Assistant](#)). And Tenable's own ServiceNow guide omits Brazil from its compatibility list ([Tenable integration guide](#)), which matters to anyone also running Tenable's CMDB connector.

## A large-enterprise program should decide keys, ownership and filters before data lands

**Pick the version track once, for every app.** USEM-only enrichment (SSVC, Early Warning), integration-health tooling, and the unified rule tables make the v30 track the forward path. ServiceNow's instruction to keep integration apps on the same side of the fork as VR, CC and CVR means the move is a coordinated program, not an app-by-app upgrade. Rehearse the Migration assistant in sub-production, and pin target versions to the KB0856498 matrix.

**Lock every key in a design review before the first production import.** Four choices are expensive or impossible to reverse:

- The VI port option cannot be turned off without deleting all VR data.
- The Wiz detection key needs a full import to change.
- Wiz CVIT keys freeze once container data exists.
- The Tenable CC test key changes how later runs match records.

For most enterprises the defensible defaults are these: VI per CI + vulnerability without port, so tasks stay readable; the Wiz UUID detection key for new deployments; a CVIT key that includes cluster or namespace only where those values map one-to-one to owning teams; and `compliance_control_id` for Tenable compliance.

**Assign scanner ownership by asset class, then enforce it.** ServiceNow keeps one VI per integration instance and has no general cross-scanner merge. A cloud VM scanned by both TVM and Wiz therefore doubles its VI count, its SLA clocks and its exception requests. A workable split is Tenable (agents and network scans) for on-premises and data-center hosts, and Wiz for cloud-native workloads, containers and cloud configuration. Use Wiz project or subscription filters, Tenable query filters and VR exclusion rules to keep each scanner inside its lane. Report remediation on CVE + CI rather than VI totals.

**Turn on the severities your policy actually governs.** Tenable TVM, Tenable.cs host and Tenable.cs container imports all default to Critical and High. If the vulnerability standard sets SLAs for Medium findings, those findings currently never exist in ServiceNow. Enable Medium in a second phase after sizing, and use the documented split-schedule pattern (criticals every four hours, the rest daily) to keep criticals current.

**Make scan cadence an SLA input, because the scanner closes the work.** A remediation owner can only Resolve. The clock stops when the next Fixed import or Wiz daily pull confirms the fix. The 90-day stale rules are too slow for ephemeral cloud assets, so add auto-close rules with shorter windows scoped to cloud discovered items. For CC, remember that staleness follows the compliance scan date.

**Choose one score and one source per signal.** KEV can arrive from the CISA feed, from Tenable's `on_cisa_kev` and from Wiz's `hasCisaKevExploit`, all landing in `cisa_exists`. EPSS can come from the CVE feed or Tenable's additional attributes. Pick the CVE-level feeds as canonical, use vendor flags as filters, and decide deliberately between the 70/15/15 Tenable Risk Rule and the composite Default Risk Rule, since the first matching rule wins. Wiz privilege and sensitive-data flags sit in `source_data` JSON and need a scripted rule to affect risk. CC risk without Service Mapping reduces to scanner criticality, and Tenable compliance contributes none.

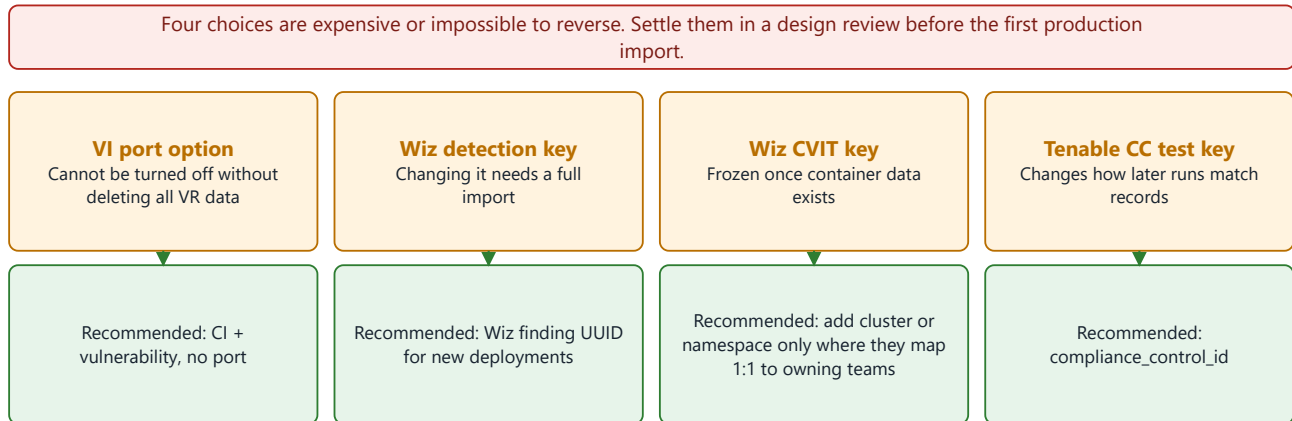
**Fix identity before go-live.** For Tenable.sc and TVM, enable network-partition lookup wherever RFC 1918 space overlaps. For Wiz, deploy the Service Graph Connector for Wiz with its own `read:resources / read:projects` service account so that cloud CIs exist before findings arrive. Set Applies to on every lookup rule. For containers, use Scanner Information unless Kubernetes Discovery reliably populates `cmdb_ci_docker_image`.

**Route Wiz CC data deliberately.** Filter the Issues integration to CLOUD\_CONFIGURATION and TOXIC\_COMBINATION, and send runtime threats to incident response. Decide whether Wiz-rejected findings should be visible (Fetch rejected) or mirrored as closures (Close rejected). Leave AI posture routing off until AI Security governance is staffed, since the switch cannot be undone.

**Engineer throughput and least privilege together.** Give TVM a Basic [16] service user with Can View, Tenable.sc a Security Analyst API key, and Wiz separate project-scoped service accounts for findings and for the CMDB, each with only the scopes of the integrations actually activated. Budget Tenable's ten concurrent exports across ServiceNow and every other consumer. Keep first-run start times to a month, and disable calculators and notifications for the initial load.

**Treat the instance, not the docs, as the specification.** Given the contradictions above, export the live transform maps, key-configuration records and integration parameters after each Store upgrade and diff them. Expect step changes in metrics around upgrades that re-key data, such as the Wiz 32.0.3 path migration.

**Figure 12. Decide these before data lands.**



Source: Official docs: vr-configure-vi-key, vr-config-wiz-host-vuln, wiz-container-tab-filters, Tenable test granularity page.

## Conclusion

The integration problem is less about connectors than about **keys and filters**. Both vendors' ServiceNow apps are now built and supported by ServiceNow. The results an enterprise sees are determined by four choices it makes once, before the first import: what makes a finding unique, which severities and resource types enter, which CMDB identity a finding resolves to, and which scanner owns an asset. Get those right and VR, CC and CVR give consistent scanner-confirmed closure. Get them wrong and the cost is duplicated work, invisible Medium risk, or keys that can no longer be changed.

The asymmetry between the vendors is also a strategic signal. Tenable's footprint narrows from three products in VR to one in CC and one in CVR: Security Center shops have no documented compliance path, and Tenable.cs container shops depend on a token model Tenable no longer documents. Wiz spans every engine from one service account, but it pulls in CC and CVR subscriptions even for host-only use. For a team running both, the effective architecture is Tenable for hosts on the network and Wiz for the cloud and containers, joined by CVE + CI reporting. ServiceNow's documentation lags its Store releases by months, so validate every assumption against a live instance.

## Appendix A. Questions to put to ServiceNow, Tenable and Wiz

### ServiceNow (get the answers in writing)

1. Which builds of the Tenable (30.x or 6.x), Wiz (32.x or 4.x) and CVR apps are certified for our family release under KB0856498?

2. Does the Wiz finding-UUID detection key apply to existing customers, and how is the required full import triggered?
3. What risk score does a Tenable compliance result receive when Tenable supplies none: the default Medium value of 20, the test criticality, or 100?
4. Which default CVIT key ships on our version, and is ServiceNow Discovery still available as a key data source?
5. How does a Wiz host finding in REJECTED status map to vulnerable-item state?
6. Which remediation-target rules ship for host vulnerable items?

## Tenable

1. What permissions does a Tenable Cloud Security API token need, now that the token page carries an end-of-support notice?
2. Does the scheduled `vpr_v2_score` deprecation affect ServiceNow's advanced filters?
3. When will Brazil appear on the compatibility list for Tenable's own ServiceNow apps?

## Wiz

1. Which scopes does the container integration need in practice, beyond `read:cloud_configuration`?
2. What API rate limits and page-size maximums apply to our tenant?
3. Which authentication and API endpoints apply to our region or government tenant?

## Appendix B. Glossary

| Term                             | Meaning                                                                                                                           |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| VR                               | Vulnerability Response: host and infrastructure vulnerabilities                                                                   |
| CC                               | Configuration Compliance: imported configuration test results                                                                     |
| CVR                              | Container Vulnerability Response: container image vulnerabilities                                                                 |
| Detection                        | One scanner-reported occurrence of a vulnerability ( <code>sn_vul_detection</code> )                                              |
| VI / VIT                         | Vulnerable item: a vulnerability on a CI ( <code>sn_vul_vulnerable_item</code> )                                                  |
| RT                               | Remediation task: a group of findings assigned for remediation                                                                    |
| TPE                              | Third-party vulnerability entry, such as a Tenable <code>TEN-&lt;plugin&gt;</code> record                                         |
| CVIT                             | Container vulnerable item ( <code>sn_vul_container_image_vulnerable_item</code> )                                                 |
| Discovered item                  | The scanner's own asset record before CI lookup ( <code>sn_sec_cm_n_src_ci</code> )                                               |
| Test group / test / result       | CC's scanner-supplied benchmark grouping, check, and pass/fail outcome                                                            |
| Authoritative source             | A standard such as CIS, DISA STIG or NIST 800-53 that a test cites                                                                |
| IRE                              | Identification and Reconciliation Engine, which creates or matches CMDB CIs                                                       |
| Tenable VM / SC / Cloud Security | Tenable Vulnerability Management (formerly Tenable.io), Tenable Security Center (Tenable.sc), Tenable Cloud Security (Tenable.cs) |
| VPR                              | Tenable Vulnerability Priority Rating                                                                                             |

| Term              | Meaning                                                                                                                          |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------|
| KEV / EPSS / SSVC | CISA Known Exploited Vulnerabilities; FIRST Exploit Prediction Scoring System; Stakeholder-Specific Vulnerability Categorization |
| USEM              | Unified Security Exposure Management: Vulnerability Response v30 and later                                                       |